

Host Cardinality & Namespace Collisions, Worked Out by Hand

One host owns N clients, each pinned to exactly one server (1:1:N). We count processes and connections, then derive when tool-name collisions appear and why prefixing eliminates them.

What this document does. The MCP host follows a strict cardinality: one host process, N in-process client objects, each bound to exactly one server. This document counts the processes and connections that implies, then treats the real hazard of aggregating N servers — tool-name collisions — as a birthday problem, and shows how server prefixing makes routing a total function.

Concepts covered: the 1:1:N pattern · process and connection counts · the aggregated namespace · collision probability (birthday) · prefixing as a routing function · injective addressing.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: one host, three servers

A host (say Claude Desktop) connects to $N = 3$ servers: `filesystem`, `github`, `postgres`. The spec's cardinality is **1:1:N**:

$$\underbrace{1}_{\text{host}} : \underbrace{1}_{\text{client per server}} : \underbrace{N}_{\text{servers}}.$$

Each client object speaks to exactly one server; reaching a second server means spawning a second client.

Intuition

“Client” in MCP is not the app — it is the small per-server protocol object *inside* the host. The host imports the client SDK and constructs one client per server, so host and clients share a process; only servers are reliably separate processes. The cardinality is what keeps every connection's state (its pending map, its negotiated capabilities) cleanly isolated to one peer.

2 Step 1 — Counting processes and connections

With N servers reached over stdio:

$$\text{processes} = \underbrace{1}_{\text{host (+ clients in-process)}} + \underbrace{N}_{\text{server subprocesses}} = 1 + N,$$

$$\text{client objects} = N, \quad \text{connections} = N \text{ (one per client).}$$

For $N = 3$: 4 processes, 3 client objects, 3 connections. Crucially the binding count is N , *not* N^2 — each client owns exactly one server, so there is no mesh inside the host.

3 Step 2 — The aggregated namespace

The host presents the LLM a single flat list of tools — the union across servers. If server i exposes t_i tools, the catalogue size is

$$T = \sum_{i=1}^N t_i.$$

Say `filesystem` exposes 8, `github` 15, `postgres` 5: $T = 8 + 15 + 5 = 28$ tools. The hazard: two servers may name a tool the same thing (both expose `search`), and a flat list cannot tell them apart.

4 Step 3 — Collisions are a birthday problem

Treat T tool names drawn from a pool of M plausible common names (`read`, `search`, `list`, `query`, ...). The probability that some two collide follows the birthday bound:

$$P(\text{collision}) \approx 1 - e^{-T(T-1)/(2M)} \approx \frac{T^2}{2M}.$$

With $T = 28$ tools over a pool of $M = 200$ common verbs:

$$P \approx 1 - e^{-28 \cdot 27 / (2 \cdot 200)} = 1 - e^{-1.89} = 1 - 0.151 = 0.849.$$

An 85% chance of *at least one* clash — collisions are the norm, not the exception, once you aggregate a handful of servers.

Mini-example — this operation in isolation

Just two servers that both expose `search`: in a flat namespace the host has two entries called `search` and no way to route a call. The LLM picks “`search`” and the host must guess which server — a 50/50 coin flip that silently sends GitHub queries to the filesystem. Even $T = 2$ can collide.

5 Step 4 — Prefixing makes routing a function

Namespace each tool by its server: `server__tool`. Now every exposed name is the pair $(\text{server}_i, \text{tool})$, which is unique by construction, so the routing map

$$\rho: \text{github_search} \mapsto \text{client}_{\text{github}} \mapsto \text{server}_{\text{github}}$$

is a well-defined (injective) function: every namespaced name routes to exactly one client, hence one server. Collision probability drops to 0 regardless of T .

Intuition

Prefixing is the same move as MCP itself (Module 0.0) and reverse-DNS extension IDs (Module 1.3): when a flat name space risks collisions, add a namespace qualifier and the space becomes a disjoint union. “Who owns this name” stops being a guess and becomes a lookup.

Watch out

Prefixing trades collisions for verbosity: the model now sees `postgres__query` instead of `query`, and overly long or cryptic prefixes hurt tool-selection accuracy. Keep server prefixes short and human — the namespace is read by the model, not just the router.

6 The argument, at a glance

#	Idea	Formula	Value ($N=3$)
1	Processes	$1 + N$	4
1	Connections (not N^2)	N	3
2	Aggregated tools	$T = \sum t_i$	28
3	Collision probability	$1 - e^{-T^2/2M}$	0.849
4	After prefixing	injective ρ	0

7 Equation cheat-sheet

Process count	$1 + N$ (host + N servers)
Client / connection count	N
Aggregated namespace	$T = \sum_i t_i$
Collision (birthday)	$1 - e^{-T(T-1)/2M} \approx T^2/2M$
Prefixed routing	$\rho(\text{srv_tool}) \rightarrow \text{one client (injective)}$

A Reference implementation

```
import math

def collision_prob(T, M):
    # birthday over name pool M
    return 1 - math.exp(-T * (T - 1) / (2 * M))

tools = {"filesystem": 8, "github": 15, "postgres": 5}
T = sum(tools.values())
# 28
print(T, round(collision_prob(T, 200), 3))
# 28 0.849

def route(namespaced):
    # server__tool -> server
    server, _, tool = namespaced.partition("__")
    return server
print(route("github__search"))
# github
```

— end of document —